
Sprawozdanie bazy danych

Mateusz

29 cze 2026

Spis treści:

1	Wstęp	1
1.1	Użyte repozytoria	1
1.2	Wnioski z przeprowadzonych ćwiczeń i eksperymentów w ramach zajęć laboratoryjnych	1
2	Badania literaturowe	3
2.1	Sprzęt dla bazy danych	3
2.2	Konfiguracja bazy danych PostgreSQL	9
2.3	Kontrola i konserwacja	19
2.4	Monitorowanie i diagnostyka	21
2.5	Wydajność, skalowanie i replikacja	25
2.6	Partycjonowanie danych	28
2.7	Bezpieczeństwo	31
2.8	Kopie zapasowe i odzyskiwanie danych	33
3	Planowanie baz danych i tworzenie dokumentacji	38
3.1	Wybór zagadnienia i opis procesów	38
3.2	Prototypowe dane	38
3.3	Model konceptualny	39
3.4	Model logiczny i normalizacja	39
3.5	Model fizyczny bazy danych	40
4	Definiowanie bazy danych i wprowadzanie danych	42
4.1	Definiowanie struktury bazy danych	42
4.2	Wybór mechanizmów wsadowego wprowadzania danych	43
4.3	Komentarz do procesu wprowadzania danych	43
5	Zapytania do bazy danych	44
5.1	Lokalizacja	44
5.2	Opis funkcji	44
5.3	Dokumentacja kodu	45
	Indeks modułów Pythona	46

Sprawozdanie stanowi podsumowanie prac laboratoryjnych z bazami danych takimi jak Postgres i SQLite na których dużo pracowaliśmy poznawaliśmy ich strukturę jak i zaawansowane techniki przetwarzania danych, a także wykorzystanie języka python do wykonywania zapytań na tych bazach danych

1.1 Użyte repozytoria

Poniżej zamieszczam linki do wykorzystywanych repozytoriów * **Moje Repozytorium:** [Sprawozdanie](#) * **Repozytoria kolegów użyte w rozdziale 2:**

- [Sprzet dla bazy danych](#)
- [Bazy danych tematyczne MK](#)
- [Bazy danych temat](#)
- [Monitoring i diagnostyka](#)
- [Tematyczne](#)
- [Partycjonowanie danych](#)
- [Kopie zapasowe](#)

1.2 Wnioski z przeprowadzonych ćwiczeń i eksperymentów w ramach zajęć laboratoryjnych

1. **Bazy danych:** Poznałem różnice między SQLite i PostgreSQL dotyczące ich działania i obsługi
2. **Python:** Nauczyłem się więcej korzystać z pythona i funkcji jakie daje w kontekście baz danych

3. **Zapytania:** Zapoznałem się z tym jak skomplikowane mogą być zapytania SQL i jak wiele zróżnicowanych opcji one dają
4. **Github:** Nauczyłem się owiele lepiej obsługiwać takie narzędzia jak github czy pakiet sphinx do generacji pdf co na pewno będzie przydatne w pisaniu pracy inżynierskiej

2.1 Sprzęt dla bazy danych

Autorzy

1. Wiktor Głogowski
2. Olga Grześkowiak
3. Kamil Karaś

2.1.1 Wstęp

Współczesne systemy zarządzania bazami danych (DBMS) stanowią fundament operacyjny większości przedsiębiorstw. Efektywność ich działania zależy bezpośrednio od synergii pomiędzy warstwą oprogramowania a architekturą sprzętową, na której są one uruchamiane. Bazy danych wykazują unikalne, skrajnie wysokie wymagania względem podsystemów wejścia/wyjścia (I/O), procesora, pamięci operacyjnej oraz niezawodności zasilania.

W zależności od profilu obciążenia systemu, infrastrukturę sprzętową klasyfikuje się pod dwa główne typy zadań:

- **OLTP (Online Transaction Processing):** Systemy transakcyjne charakteryzujące się ogromną liczbą jednoczesnych, krótkich operacji zapisu i odczytu (np. systemy bankowe, sklepy internetowe). Kluczowym parametrem jest tu minimalne opóźnienie oraz wysoka liczba operacji wejścia/wyjścia na sekundę (IOPS).
- **OLAP (Online Analytical Processing):** Hurtownie danych i systemy analityczne realizujące długie, złożone zapytania, które agregują miliony rekordów. Tutaj priorytetem jest przepustowość sekwencyjna oraz masowa wielordzeniowość procesorów.

Poniższy rozdział opisuje architekturę fizyczną bazy danych oraz wytyczne w sprawie doboru komponentów.

2.1.2 Procesor (CPU)

Wybór procesora dla serwera bazodanowego determinowany jest nie tylko surową wydajnością obliczeniową, ale również architekturą pamięci podręcznej, topologią połączeń międzygniazdowych oraz modelami licencjonowania oprogramowania.

Wydajność Jednowątkowa vs Wielordzeniowość

Decyzja o wyborze modelu procesora zależy od struktury zapytań generowanych przez aplikację:

1. **Taktowanie Zegara:** W środowiskach OLTP, gdzie transakcje często wykonują się sekwencyjnie lub są blokowane przez zamki i zatraski, wyższe taktowanie rdzenia (np. 3.6 GHz - 4.0 GHz) przynosi lepsze rezultaty niż ekstremalna liczba rdzeni o niskim taktowaniu. Szybka realizacja pojedynczego wątku skraca czas trwania blokad.
2. **Skalowanie Wielordzeniowe:** W systemach OLAP silniki baz danych potrafią dzielić jedno zapytanie na wiele wątków. W tym przypadku procesory posiadające 64, 96 lub więcej rdzeni fizycznych (np. AMD EPYC, Intel Xeon Scalable) pozwalają na równoległe skanowanie indeksów i tabel, drastycznie przyspieszając operacje analityczne.

Wpływ Licencjonowania na Wybór CPU: Wiodący producenci oprogramowania bazodanowego (np. Microsoft SQL Server Enterprise, Oracle) stosują model licencjonowania per-core. Koszt licencji na jeden rdzeń wielokrotnie przewyższa koszt zakupu samego sprzętu. Z tego powodu ekonomicznie i wydajnościowo uzasadniony jest wybór procesorów o mniejszej liczbie rdzeni, ale o maksymalnym dostępnym taktowaniu bazowym i zaawansowanej architekturze cache.

Topologia NUMA (Non-Uniform Memory Access)

Wieloprocessorowe serwery (np. architektury dwu- lub czterogniazdowe) dzielą zasoby na tzw. węzły NUMA. Każdy procesor posiada swój zintegrowany kontroler pamięci i bezpośredni dostęp do fizycznie najbliższych banków RAM (pamięć lokalna).

- **Dostęp Lokalny:** Charakteryzuje się minimalnymi opóźnieniami i maksymalnym pasmem przenoszenia.
- **Dostęp Zdalny:** Jeśli procesor osadzony w gnieździe 0 potrzebuje danych znajdujących się w banku pamięci przypisanym do gniazda 1, transakcja musi odbyć się za pośrednictwem magistrali międzyprocesorowej (Intel UPI lub AMD Infinity Fabric). Generuje to narzut opóźnień zwany *NUMA penalty*.

Niewłaściwa alokacja wątków przez system operacyjny może prowadzić do ciągłego przemieszczania danych między węzłami (*NUMA bouncing*), co drastycznie obniża wydajność bazy danych. Wymagane jest stosowanie silników bazodanowych w pełni wspierających i optymalizujących architekturę NUMA (*NUMA-aware*).

Znaczenie Pamięci Podręcznej (Cache L3)

Struktury danych w relacyjnych bazach (drzewa B+, tabele stron) sprawiają, że procesor wykonuje ogromną liczbę operacji skoków i wyszukiwań wskaźników. Powoduje to częste chybienia pamięci podręcznej, zmuszając procesor do czekania na dane z RAM-u. Procesory wyposażone w powiększoną pamięć podręczną poziomu trzeciego (L3), takie jak układy z technologią 3D V-Cache, pozwalają na zatrzymanie krytycznych struktur indeksowych wewnątrz struktur procesora, generując ogromne zyski wydajnościowe.

2.1.3 Pamięć RAM

Pamięć operacyjna RAM decyduje o tym, jak duża część bazy danych może być przetwarzana bezpośrednio w super-szybkiej pamięci półprzewodnikowej, bez konieczności odwoływania się do podsystemu dyskowego.

Mechanizm Buforowania Danych

Podstawową jednostką wymiany danych między dyskiem a pamięcią w bazach danych jest strona (zazwyczaj o rozmiarze 8 KB). Silnik bazy danych alokuje obszar pamięci RAM nazywany **Buffer Pool** (w MS SQL Server) lub **Shared Buffers** (w PostgreSQL).

- **Odczyty z Pamięci (Logical Reads):** Jeśli żądana strona danych znajduje się już w RAM, odczyt następuje natychmiastowo (rzędu nanosekund).
- **Odczyty z Dysku (Physical Reads):** W przypadku braku strony w buforze, wątek bazy danych zostaje zawieszony do czasu fizycznego załadowania strony z pamięci masowej, co zwiększa czas odpowiedzi o rzędy wielkości.

Docelowym stanem konfiguracyjnym dla systemów produkcyjnych jest posiadanie takiej ilości pamięci RAM, która pozwala na bezproblemowe przetrzymywanie w całości tzw. hot data set, czyli najczęściej modyfikowanych i odpytywanych danych wraz z ich indeksami.

Wymóg Technologii ECC (Error-Correcting Code)

W serwerach bazodanowych niedopuszczalne jest stosowanie pamięci bez obsługi korekcji błędów (non-ECC). Pamięci **ECC** posiadają dodatkowe układy scalone pozwalające na wykrywanie i automatyczną korekcję błędów jednobitowych oraz wykrywanie błędów wielobitowych.

Wystąpienie zjawiska *bit-flip* (samorzutnej zmiany wartości bitu w pamięci RAM pod wpływem np. tła radiacyjnego lub zakłóceń elektromagnetycznych) w systemie bez ECC niesie katastrofalne skutki dla baz danych:

- **Cicha korupcja danych:** Błędna wartość może zostać zapisana z bufora pamięci wprost na dysk, powodując trwałe i niezauważalne zniszczenie struktury tabeli lub błędne zapisy księgowe.
- **Uszkodzenie stron indeksowych:** Powoduje błędy integralności i uniemożliwia poprawne wykonywanie zapytań SQL.
- **Awaria krytyczna:** W przypadku naruszenia pamięci jądra systemu operacyjnego lub krytycznego procesu bazy danych dochodzi do natychmiastowego zatrzymania serwera (Kernel Panic / BSOD).

Parametry Wydajnościowe: Przepustowość i Opóźnienia

Współczesne standardy serwerowe opierają się na pamięciach DDR5, oferujących zaawansowaną architekturę wielokanałową (np. 8 lub 12 kanałów pamięci na jedno gniazdo CPU). Wysoka przepustowość (wyrażana w MT/s) przyspiesza operacje sortowania w pamięci, tworzenie tabel tymczasowych oraz operacje złączania typu *Hash Join*, które są na porządku dziennym w systemach analitycznych.

2.1.4 Pamięć Masowa

Podsystem pamięci masowej bezpośrednio determinuje ostateczną trwałość danych (cecha *Durability* z paradygmatu ACID) oraz jest najczęstszym wąskim gardłem systemu informatycznego z uwagi na mechaniczne lub interfejsowe ograniczenia prędkości.

Klasyfikacja Nośników i Ich Przydatność

Współczesna inżynieria bazodanowa całkowicie odchodzi od talerzowych dysków magnetycznych HDD w środowiskach produkcyjnych na rzecz pamięci półprzewodnikowych Flash.

- **Dyski HDD (SAS 10K/15K):** Ze względu na ograniczenia mechaniczne oferują zaledwie kilkaset IOPS i wysokie opóźnienia (>5 ms). Ich zastosowanie ogranicza się obecnie wyłącznie do składowania archiwalnych kopii zapasowych (backups).
- **Dyski SSD (SATA/SAS Enterprise):** Zapewniają stabilną wydajność na poziomie do 90 000 IOPS dla odczytu, jednak ograniczeniem staje się interfejs SATA (6 Gb/s) lub SAS-3 (12 Gb/s).
- **Nośniki NVMe (PCIe Gen4/Gen5):** Komunikują się bezpośrednio z procesorem poprzez magistralę PCI Express, eliminując narzut protokołów SCSI/ATA. Osiągają wydajność przekraczającą 1 000 000 IOPS oraz redukują opóźnienia do wartości mikrosekundowych (<50 μ s), stając się bezdyskusyjnym standardem dla baz danych.

Fizyczna Separacja Architektury Plików

Poprawna konfiguracja pamięci masowej wymaga rozdzielenia różnych typów aktywności wejścia/wyjścia na odseparowane fizycznie wolumeny dyskowe:

1. **Dziennik Transakcji:** Każda operacja modyfikacji danych (INSERT, UPDATE, DELETE) musi zostać najpierw sekwencyjnie zapisana w dzienniku transakcji, zanim baza potwierdzi jej pomyślne wykonanie. Ruch ten ma charakter niemal wyłącznie sekwencyjnego zapisu o krytycznym znaczeniu dla opóźnienia aplikacji. Dziennik transakcji powinien znajdować się na najszybszych dedykowanych nośnikach o minimalnym opóźnieniu zapisu.
2. **Pliki Danych:** Przechowują właściwe tabele i indeksy. Ruch na tych plikach ma charakter wysoce losowy (zarówno odczyty, jak i zapisy wywoływane przez proces *Checkpoint*, który zrzuca zmodyfikowane strony z RAM na dysk). Wymagają macierzy zoptymalizowanej pod kątem wysokiej liczby losowych IOPS.
3. **Baza Tymczasowa (TempDB / Swapping Space):** Wykorzystywana przez silnik bazy do przechowywania obiektów tymczasowych, wyników pośrednich podzapytań oraz sor-

towania. Generuje bardzo intensywny, mieszany ruch. Umieszcza się ją na ultra-szybkich dyskach o wysokiej wytrzymałości.

Żywotność i Wytrzymałość Zapisu (DWPD)

Bazy danych nieustannie modyfikują i nadpisują sektory. Z tego powodu stosowanie dysków klasy konsumenckiej grozi ich błyskawicznym zużyciem. Przy doborze nośników kluczowym parametrem jest **DWPD** (*Drive Writes Per Day*) – wskaźnik określający, ile razy dziennie można zapisać całą pojemność dysku przez okres gwarancyjny (zazwyczaj 5 lat). Do baz danych wymagane są nośniki klasy *Enterprise Mixed-Use* lub *Write-Intensive* o współczynniku DWPD wynoszącym minimum 3-5.

2.1.5 Kontrolery i Interfejsy Pamięci

Kontrolery realizują zadania pośrednictwa, zarządzania macierzami nadmiarowymi (RAID) oraz gwarantują integralność zapisu.

Sprzętowy RAID vs Software-Defined Storage

Przez lata standardem były sprzętowe kontrolery RAID (*Hardware RAID*) wyposażone we własny procesor i pamięć cache. Odciażają one procesor główny serwera z obliczeń sum kontrolnych (szczególnie parzystości w RAID 5/6).

W przypadku nowoczesnych pamięci NVMe sprzętowe kontrolery starszego typu stawały się wąskim gardłem, blokując bezpośredni dostęp do linii PCIe. Współcześnie odchodzi się od nich na rzecz:

- **Bezpośredniego podłączenia NVMe (Direct Attached PCIe)** i zarządzania programowego na poziomie zaawansowanych systemów plików (np. ZFS) lub warstw logicznych OS (np. Linux mdadm / LVM).
- Modernistycznych, dedykowanych sprzętowych kontrolerów NVMe RAID klasy Enterprise, zdolnych do natywnego przetwarzania protokołu NVMe z pełną prędkością magistrali PCIe Gen5.

Pamięć Podtrzymywana Bateriajnie (BBU/FBWC): W przypadku korzystania ze sprzętowego kontrolera RAID z włączoną pamięcią podręczną zapisu (*Write-Back Cache*), **bezwzględny warunkiem bezpieczeństwa** jest obecność modułu **BBU** (*Battery Backup Unit*) lub **FBWC** (*Flash-Backed Write Cache*). W razie awarii zasilania serwera, moduł ten podtrzymuje dane w pamięci cache kontrolera (lub przepisuje je do dedykowanej pamięci flash), zapobiegając utracie nieutralizowanych transakcji i uszkodzeniu struktury macierzy.

Wybór Poziomu RAID pod Kątem Baz Danych

- **RAID 10 (Striped Mirror):** Rekomendowany układ dla większości systemów bazodanych. Łączy zalety wydajnościowe paskowania (RAID 0) i bezpieczeństwa lustrzanego (RAID 1). Zapewnia doskonałą wydajność losowego zapisu, ponieważ nie generuje tzw. narzutu parzystości (*write penalty*).
- **RAID 5 / RAID 6:** Zapewniają lepsze wykorzystanie przestrzeni dyskowej, ale każda operacja zapisu wymaga odczytu danych, obliczenia nowej parzystości i ponownego zapisu (*kara za zapis*). Są **wysoce niewskazane dla baz transakcyjnych OLTP**. Mogą być

stosowane jedynie w hurtowniach danych (OLAP), gdzie operacje zapisu są sekwencyjne i rzadkie (procesy ETL).

2.1.6 Zasilanie i Niezawodność

Podsystem zasilania w infrastrukturze bazodanowej traktowany jest jako integralny element ochrony integralności logicznej danych, a nie tylko komponent ciągłości działania.

Nadmiarowość Zasilaczy i Niezależne Linie Zasilające

Serwer bazodanowy musi posiadać zasilacze nadmiarowe typu *Hot-Swap* (z możliwością wymiany podczas pracy) w konfiguracji minimum **N+1** lub optymalnie **2N** (Full Redundancy).

Zasilacze muszą być podpięte do fizycznie odseparowanych źródeł zasilania:

- **Zasilacz A:** Podłączony do Linii zasilającej A oraz dedykowanego modułu UPS A.
- **Zasilacz B:** Podłączony do Linii zasilającej B oraz dedykowanego modułu UPS B.

Taka topologia gwarantuje, że awaria jednego zasilacza, uszkodzenie kabla lub całkowity zanik napięcia na jednej z linii energetycznych nie spowoduje wyłączenia serwera.

Infrastruktura UPS i Agregaty Prądotwórcze

Wszelkie systemy serwerowe DBMS muszą współpracować z zasilaczami awaryjnymi UPS działającymi w topologii *Online* (podwójne przetwarzanie). Zapewniają one idealną sinusoidę napięcia, eliminują przepięcia i gwarantują czas podtrzymania niezbędny do automatycznego uruchomienia spalinowych agregatów prądotwórczych o odpowiedniej mocy.

Zapobieganie Zjawisku „Torn Writes”

Zjawisko *Torn Write* (rozbity/częściowy zapis) występuje, gdy system operacyjny przesyła do zapisu stronę danych (np. 8 KB), a w trakcie fizycznego zapisu na komórkę pamięci Flash dochodzi do nagłego odcięcia zasilania. W efekcie na dysku ląduje np. tylko 4 KB danych, co powoduje bezpowrotne uszkodzenie struktury tej strony i błąd sumy kontrolnej przy próbie kolejnego uruchomienia bazy danych.

Poza mechanizmami programowymi (takimi jak *Doublewrite Buffer* w silniku InnoDB/MySQL), na poziomie sprzętowym stosuje się technologię **PLP (Power Loss Protection)**. Dyski klasy Enterprise posiadają wbudowane banki kondensatorów tantalowych. W momencie wykrycia spadku napięcia, energia z kondensatorów pozwala na podtrzymanie pracy kontrolera dysku przez czas wymagany do bezpiecznego opróżnienia pamięci podręcznej RAM dysku i utrwalenia wszystkich danych w nieulotnych komórkach pamięci NAND Flash.

2.1.7 Podsumowanie

Dobór sprzętu dla systemów zarządzania bazami danych musi być ściśle dopasowany do profilu obciążenia, z wyraźnym podziałem na transakcyjne środowiska OLTP oraz analityczne systemy OLAP. Wybór procesora stanowi kompromis pomiędzy taktowaniem a wielordzeniowością ze względu na koszty licencyjne, podczas gdy w przypadku pamięci operacyjnej absolutnym wymogiem jest zastosowanie technologii ECC chroniącej przed cichą korupcją danych. W obszarze

pamięci masowej standardem stały się nośniki półprzewodnikowe NVMe o wysokiej wytrzymałości (DWPD), które ze względów wydajnościowych wymagają fizycznej separacji wolumenów dla dzienników transakcji i plików danych. Do zabezpieczenia pracy dysków rekomenduje się użycie konfiguracji RAID 10 w połączeniu z nowoczesnymi kontrolerami sprzętowymi lub rozwiązaniami programowymi. Całość infrastruktury musi być kategorycznie chroniona przed awariami poprzez nadmiarowe zasilacze w konfiguracji 2N, systemy UPS w topologii Online oraz dyski wyposażone w sprzętową ochronę PLP eliminującą zjawisko uszkodzonych zapisów.

2.2 Konfiguracja bazy danych PostgreSQL

2.2.1 Wstęp

Rozdział opisuje konfigurację połączenia z bazą danych **PostgreSQL** – dojrzałym, open-source’owym systemem zarządzania relacyjnymi bazami danych (RDBMS), który wyróżnia się pełną zgodnością ze standardem SQL oraz silnym naciskiem na rozszerzalność i niezawodność.

Wymagania:

- dostęp do instancji PostgreSQL,
- zmienne środowiskowe,
- narzędzia projektowe (np. `psql`, `pg_isready`).

2.2.2 Podstawowe parametry

Każde połączenie z PostgreSQL wymaga:

- hosta i portu (domyślnie `localhost` i `5432`),
- nazwy bazy danych,
- użytkownika i hasła.

Dane wrażliwe (np. hasła) przechowuj w **zmiennych środowiskowych** – to oddziela konfigurację od kodu. Pamiętaj: zmienne środowiskowe nie są mechanizmem bezpieczeństwa; w produkcji uzupełnij je o dedykowane zarządzanie sekretami (np. HashiCorp Vault, AWS Secrets Manager).

2.2.3 Lokalizacja i struktura katalogów PostgreSQL

Pliki konfiguracyjne PostgreSQL znajdują się w katalogu danych (`PGDATA`).

Główne pliki konfiguracyjne:

- `postgresql.conf` – podstawowa konfiguracja serwera (port, pamięć, logi)
- `pg_hba.conf` – kontrola dostępu (kto i skąd może się łączyć)
- `pg_ident.conf` – mapowanie użytkowników systemowych na role PostgreSQL

Domyślne lokalizacje:

Tabela 1: Domyślne lokalizacje

System	Katalog danych
Linux (RPM/DEB)	/var/lib/pgsql/data/ lub /var/lib/postgresql/*/main/
Linux (kompilacja źródłowa)	/usr/local/pgsql/data/
Windows	C:\Program Files\PostgreSQL\<version>\data\
Docker	/var/lib/postgresql/data/ (w kontenerze)

Zmiana katalogu danych:

```
# Inicjalizacja nowego katalogu
initdb -D /ścieżka/do/katalogu

# Uruchomienie z niestandardowym PGDATA
pg_ctl -D /ścieżka/do/katalogu start
```

Struktura katalogu danych:

```
$PGDATA/
├── postgresql.conf # główny plik konfiguracyjny
├── pg_hba.conf      # polityka dostępu
├── pg_ident.conf    # mapowanie tożsamości
├── base/            # rzeczywiste bazy danych
├── global/          # tabele systemowe
├── log/             # logi serwera
└── pg_wal/          # Write-Ahead Log (WAL)
```

Sprawdzenie aktualnej lokalizacji:

```
SHOW data_directory;
SHOW config_file;
SHOW hba_file;
```

2.2.4 Środowiska i profile

Przełączanie środowisk PostgreSQL (dev/test/prod) realizuj przez:

- zmienne środowiskowe (np. PGHOST, PGPORT, PGDATABASE, PGUSER, PGPASSWORD),
- profile konfiguracyjne,
- osobne pliki .env dla każdego środowiska.

2.2.5 Automatyzacja

Zalecenia dla PostgreSQL:

- walidacja konfiguracji przy starcie za pomocą `pg_isready`,
- skrypty sprawdzające kompletność zmiennych środowiskowych,
- integracja z CI/CD (np. GitHub Actions, GitLab CI).

Dokumentację generuj automatycznie (Sphinx).

2.2.6 Przykłady

PostgreSQL – zmienne środowiskowe (.env)

```
PGHOST=localhost
PGPORT=5432
PGDATABASE=aplikacja
PGUSER=app_user
PGPASSWORD=${DB_PASSWORD}
```

PostgreSQL – URL połączenia

```
DATABASE_URL=postgresql://${PGUSER}:${PGPASSWORD}@${PGHOST}:${PGPORT}/${PGDATABASE}
```

PostgreSQL – sprawdzenie połączenia

```
pg_isready -h ${PGHOST} -p ${PGPORT} -U ${PGUSER} -d ${PGDATABASE}
```

2.2.7 Najlepsze praktyki

1. Nie przechowuj sekretów w repozytorium.
2. Stosuj `.env.example` zamiast rzeczywistych danych.
3. Używaj standardowych zmiennych środowiskowych PostgreSQL (PG*).
4. Waliduj konfigurację przed uruchomieniem (`pg_isready`).
5. W środowiskach produkcyjnych używaj połączeń SSL/TLS (`sslmode=require`).

2.2.8 Podsumowanie

Poprawna konfiguracja połączenia z PostgreSQL zwiększa bezpieczeństwo i przenośność aplikacji między środowiskami. Standardowe zmienne środowiskowe oraz narzędzia takie jak `pg_isready` ułatwiają automatyzację i walidację.

2.2.9 Szczegółowe omówienie plików konfiguracyjnych

Po omówieniu podstawowych wymagań środowiskowych można przejść do szczegółowej konfiguracji PostgreSQL.

W kolejnych sekcjach przedstawiono najważniejsze parametry konfiguracyjne dostępne w plikach:

- `postgresql.conf` – konfiguracja działania serwera,
- `pg_hba.conf` – kontrola dostępu do bazy danych,
- `pg_ident.conf` – mapowanie użytkowników systemowych na role PostgreSQL.

2.2.10 `postgresql.conf`

Plik `postgresql.conf` jest głównym plikiem konfiguracyjnym klastra PostgreSQL. Zawiera ustawienia wpływające na działanie serwera, wydajność, bezpieczeństwo, logowanie oraz replikację.

Lokalizację aktywnego pliku można sprawdzić poleceniem:

```
SHOW config_file;
```

Najważniejsze parametry

Połączenia sieciowe

`listen_addresses`

Określa adresy IP, na których PostgreSQL nasłuchuje połączeń.

Najczęstsze wartości:

- `localhost` – tylko połączenia lokalne,
- `*` – wszystkie interfejsy sieciowe,
- lista adresów, np. `'192.168.1.10,localhost'`.

`port`

Port TCP używany przez PostgreSQL.

Domyślna wartość:

```
port = 5432
```

`max_connections`

Maksymalna liczba jednoczesnych połączeń z bazą danych.

Zbyt wysoka wartość może zwiększać zużycie pamięci.

Pamięć i wydajność

shared_buffers

Ilość pamięci RAM używanej przez PostgreSQL do buforowania danych.

Typowe wartości:

- 128MB – środowiska testowe,
- 25% pamięci RAM – środowiska produkcyjne.

work_mem

Ilość pamięci wykorzystywanej przez operacje sortowania i zapytania.

Parametr jest przydzielany dla pojedynczej operacji.

maintenance_work_mem

Pamięć używana podczas operacji administracyjnych, np.:

- VACUUM,
- CREATE INDEX,
- ALTER TABLE.

effective_cache_size

Szacowany rozmiar pamięci podręcznej dostępnej dla PostgreSQL.

Parametr pomaga optymalizatorowi zapytań wybierać odpowiednie plany wykonania.

Logowanie

logging_collector

Włącza zapisywanie logów do plików.

Dostępne wartości:

- on,
- off.

log_directory

Katalog przechowywania logów.

log_filename

Wzorzec nazw plików logów.

Przykład:

```
log_filename = 'postgresql-%Y-%m-%d.log'
```

log_min_duration_statement

Określa minimalny czas wykonania zapytania (w ms), po którym zostanie ono zapisane w logach.

Przykład:

```
log_min_duration_statement = 1000
```

Wartość 1000 oznacza logowanie zapytań trwających dłużej niż 1 sekundę.

Bezpieczeństwo

password_encryption

Algorytm szyfrowania haseł użytkowników.

Zalecana wartość:

```
password_encryption = scram-sha-256
```

ssl

Włącza obsługę połączeń SSL/TLS.

Dostępne wartości:

- on,
- off.

ssl_cert_file / ssl_key_file

Ścieżki do certyfikatu i klucza prywatnego SSL.

Replikacja i WAL

wal_level

Określa poziom szczegółowości Write-Ahead Log (WAL).

Dostępne wartości:

- minimal,
- replica,
- logical.

max_wal_size

Maksymalny rozmiar plików WAL przed wykonaniem checkpointu.

archive_mode

Włącza archiwizację WAL.

archive_command

Polecenie wykonywane podczas archiwizacji plików WAL.

Przykładowa konfiguracja

```
listen_addresses = '*'
port = 5432
max_connections = 200
```

(ciąg dalszy na następnej stronie)

(kontynuacja poprzedniej strony)

```
shared_buffers = 1GB
work_mem = 16MB
maintenance_work_mem = 256MB

logging_collector = on
log_directory = 'log'

ssl = on
password_encryption = scram-sha-256

wal_level = replica
max_wal_size = 2GB
```

Weryfikacja konfiguracji

Po zmianie parametrów konfigurację można przeładować bez restartu serwera:

```
SELECT pg_reload_conf();
```

Niektóre parametry wymagają pełnego restartu PostgreSQL.

Aktualne wartości parametrów można sprawdzić poleceniem:

```
SHOW shared_buffers;
SHOW wal_level;
SHOW max_connections;
```

2.2.11 pg_hba.conf

Plik `pg_hba.conf` (Host-Based Authentication) odpowiada za kontrolę dostępu do serwera PostgreSQL.

Definiuje:

- kto może połączyć się z bazą danych,
- z jakiego adresu IP,
- do jakiej bazy danych,
- przy użyciu jakiej metody uwierzytelniania.

Lokalizację aktywnego pliku można sprawdzić poleceniem:

```
SHOW hba_file;
```

Składnia wpisu

Każdy wpis w `pg_hba.conf` ma następującą strukturę:

TYPE	DATABASE	USER	ADDRESS	METHOD
------	----------	------	---------	--------

Znaczenie pól:

TYPE

Typ połączenia.

Dostępne wartości:

- `local` – połączenia lokalne (Unix socket),
- `host` – połączenia TCP/IP,
- `hostssl` – połączenia TCP/IP z SSL,
- `hostnoss` – połączenia TCP/IP bez SSL.

DATABASE

Nazwa bazy danych, do której użytkownik może się połączyć.

Możliwe wartości:

- konkretna nazwa bazy,
- `all` – wszystkie bazy,
- `sameuser` – baza o nazwie zgodnej z użytkownikiem.

USER

Użytkownik lub rola PostgreSQL.

ADDRESS

Adres IP lub zakres sieci w notacji CIDR.

Przykłady:

- `127.0.0.1/32`,
- `192.168.1.0/24`,
- `0.0.0.0/0` – wszystkie adresy.

METHOD

Metoda uwierzytelniania użytkownika.

Najczęściej używane metody uwierzytelniania

trust

Zezwala na połączenie bez uwierzytelniania.

Zalecane wyłącznie w środowiskach testowych.

reject

Odrzuca połączenie.

md5

Uwierzytelnianie przy użyciu hasła MD5.

Metoda starsza i mniej bezpieczna niż SCRAM.

scram-sha-256

Nowoczesna metoda uwierzytelniania oparta na SCRAM.

Zalecana dla nowych instalacji PostgreSQL.

peer

Uwierzytelnianie na podstawie użytkownika systemowego.

Najczęściej używane dla lokalnych połączeń Linux/Unix.

ident

Mapowanie użytkownika systemowego przy użyciu `pg_ident.conf`.

Przykładowa konfiguracja

```
# Połączenia lokalne
local all postgres peer

# Połączenia localhost IPv4
host all all 127.0.0.1/32 scram-sha-256

# Połączenia localhost IPv6
host all all ::1/128 scram-sha-256

# Sieć lokalna
host aplikacja app_user 192.168.1.0/24 scram-sha-256
```

Najlepsze praktyki

1. Używaj `scram-sha-256` zamiast `md5`.
2. Ograniczaj dostęp do konkretnych adresów IP.
3. Nie używaj `trust` w środowiskach produkcyjnych.
4. Stosuj `hostssl` dla połączeń zdalnych.
5. Po zmianie konfiguracji wykonuj `reload` konfiguracji:

```
SELECT pg_reload_conf();
```

2.2.12 pg_ident.conf

Plik `pg_ident.conf` umożliwia mapowanie użytkowników systemowych na role PostgreSQL.

Najczęściej używany jest razem z metodami uwierzytelniania:

- `peer`,

- ident.

Lokalizację aktywnego pliku można sprawdzić poleceniem:

```
SHOW ident_file;
```

Zastosowanie

Mechanizm mapowania pozwala określić, który użytkownik systemowy może zostać powiązany z konkretną rolą PostgreSQL.

Jest to szczególnie przydatne:

- w środowiskach Linux/Unix,
- przy automatyzacji administracyjnej,
- dla lokalnych połączeń bez użycia haseł.

Składnia wpisu

MAPNAME	SYSTEM-USERNAME	PG-USERNAME
---------	-----------------	-------------

Znaczenie pól:

MAPNAME

Nazwa mapowania wykorzystywana w `pg_hba.conf`.

SYSTEM-USERNAME

Użytkownik systemu operacyjnego.

PG-USERNAME

Rola PostgreSQL, na którą użytkownik ma zostać zmapowany.

Przykładowa konfiguracja

# MAPNAME	SYSTEM-USERNAME	PG-USERNAME
admin_map	postgres	postgres
admin_map	deploy	db_admin
app_map	appuser	aplikacja

Przykład użycia w `pg_hba.conf`:

```
local    all        all        peer map=admin_map
```

W powyższym przykładzie użytkownicy systemowi zdefiniowani w `admin_map` mogą logować się jako odpowiadające im role PostgreSQL.

Najlepsze praktyki

1. Ograniczaj mapowania wyłącznie do wymaganych użytkowników.
2. Nie mapuj użytkowników systemowych na role superuser bez uzasadnienia.
3. Stosuj osobne role administracyjne i aplikacyjne.
4. Dokumentuj wszystkie mapowania użytkowników.
5. Regularnie przeglądaj konfigurację dostępu.

Opracowanie

Autor: Michał Kraus **Przedmiot:** Bazy danych **Data:** maj 2026

2.3 Kontrola i konserwacja

Autorzy

1. Paweł Łoćwin
2. Paweł Łosowski

2.3.1 Wprowadzenie i planowanie konserwacji

Współczesne systemy relacyjnych baz danych to wysoce skomplikowane środowiska, które wymagają ciągłego i proaktywnego nadzoru. Aby zagwarantować wysoką dostępność i niezawodność, niezbędne jest rygorystyczne planowanie procesów konserwacyjnych, które opiera się na odpowiedzi na trzy kluczowe pytania:

- **Kiedy?** Operacje inwazyjne (np. pełna przebudowa tabel i indeksów) powinny być planowane w tzw. oknach serwisowych, czyli w godzinach najniższego obciążenia systemu (zwykle w nocy lub w weekendy). Lżejsze prace konserwacyjne mogą odbywać się w tle.
- **W jakim zakresie?** Należy precyzyjnie określić, czy konserwacji wymaga cała instancja, pojedyncza baza danych, czy tylko wytypowane, najszybciej rosnące tabele. Skupienie się na najbardziej obciążonych obszarach oszczędza zasoby sprzętowe.
- **Jak?** Konserwacja powinna być maksymalnie zautomatyzowana (np. z wykorzystaniem systemowego demona CRON lub rozszerzenia *pg_cron*). Ręczne interwencje administratora powinny być zarezerwowane wyłącznie dla sytuacji awaryjnych i niestandardowych problemów wydajnościowych.

2.3.2 Zarządzanie stanem serwera i sesjami

Podstawą kontroli nad środowiskiem bazodanowym jest zarządzanie cyklem życia samej usługi. Uruchamianie, zatrzymywanie i restartowanie serwera bazy danych realizuje się najczęściej za pośrednictwem menedżera usług systemu operacyjnego (np. polecenia *systemctl start/stop/restart postgresql*) lub za pomocą dedykowanego narzędzia wiersza poleceń *pg_ctl*.

Podczas wdrażania krytycznych aktualizacji lub w sytuacjach awaryjnych, konieczne jest sprawne zarządzanie ruchem użytkowników:

- **Zapobieganie nowym połączeniom:** Administrator może tymczasowo zablokować możliwość logowania się do konkretnej bazy danych poprzez zmianę jej metadanych. Pozwala to na „odcięcie” aplikacji bez wyłączania całego serwera.
- **Ograniczanie i rozłączanie użytkowników:** Istniejące, aktywne sesje, które np. zawiesiły się lub blokują ważne operacje, można siłowo przerwać, zwalniając tym samym zablokowane zasoby.

```
-- Zapobieganie nawiązywaniu nowych połączeń do bazy danych
ALTER DATABASE biblioteka_db ALLOW_CONNECTIONS = false;

-- Siłowe rozłączenie wszystkich aktywnych użytkowników (poza aktualną
-- sesją)
SELECT pg_terminate_backend(pid)
FROM pg_stat_activity
WHERE datname = 'biblioteka_db' AND pid <> pg_backend_pid();
```

2.3.3 Proces Vacuum

Architektura PostgreSQL opiera się na mechanizmie MVCC (Multi-Version Concurrency Control). Oznacza to, że instrukcje aktualizacji (UPDATE) i usuwania (DELETE) nie niszczą fizycznie starych danych, lecz tworzą tzw. „martwe krotki” (dead tuples). Do zarządzania nimi służy proces czyszczenia:

- **VACUUM:** Standardowa operacja zwalniająca miejsce po martwych wierszach, czyniąca je dostępnymi dla nowych danych. Nie zmniejsza ona fizycznego rozmiaru pliku na dysku i nie blokuje możliwości jednoczesnego odczytu i zapisu tabeli.
- **VACUUM FULL:** Inwazyjna i ciężka operacja, która fizycznie przebudowuje tabelę, faktycznie zmniejszając jej rozmiar i zwracając wolne miejsce do systemu operacyjnego. Wymaga ekskluzywnej blokady (lock), całkowicie odcinając aplikację od modyfikowania tabeli na czas trwania procesu.
- **AUTOVACUUM:** Zautomatyzowany proces działający w tle, który regularnie monitoruje poziom zmian w tabelach i samoczynnie wyzwała standardowy Vacuum, zapobiegając niekontrolowanemu puchnięciu bazy danych (table bloat).

2.3.4 Zarządzanie transakcjami i schematy (SCHEMA)

Kolejnym aspektem administracji jest dbanie o logiczną strukturę bazy oraz spójność operacji.

SCHEMA (Schematy) to wirtualne przestrzenie nazw (przypominające katalogi w systemie operacyjnym), służące do logicznego grupowania tabel, widoków i funkcji. Ich główne zastosowania to: * Izolacja danych różnych mikrousług lub aplikacji wewnątrz jednej wspólnej bazy danych. * Implementacja architektury wielodostępowej (multi-tenant), gdzie każdy klient aplikacji obsługiwany jest w dedykowanym, oddzielnym schemacie. * Ułatwienie i zbiorcze zarządzanie uprawnieniami dostępu.

Zarządzanie transakcjami to proces kontrolowania instrukcji (BEGIN, COMMIT, ROLLBACK) w taki sposób, aby zachować zgodność z regułami ACID. Administrator musi zwracać szczególną uwagę na zapobieganie tzw. długim transakcjom (long-running transactions). Otwarta i porzucona transakcja powstrzymuje proces Vacuum przed usuwaniem martwych krotek, co prowadzi do drastycznego spadku wydajności całej bazy.

```
-- Przykład logiki transakcyjnej z wykorzystaniem konkretnego schematu
BEGIN;
INSERT INTO finanse.historia (kwota, opis) VALUES (200, 'Opłata_
→serwisowa');
UPDATE finanse.konta SET saldo = saldo - 200 WHERE id_klienta = 5;
COMMIT;
```

2.3.5 Zarządzanie indeksami

Indeksy są fundamentalne dla wydajnego wyszukiwania danych, jednak ich utrzymanie kosztuje — spowalniają one modyfikacje danych (DML) i konsumują miejsce na dysku.

Prawidłowe zarządzanie indeksami polega na stałym monitorowaniu ich wykorzystania. Należy identyfikować i usuwać indeksy, które nie są wykorzystywane przez planistę zapytań (tzw. unused indexes). Dodatkowo, podobnie jak tabele, indeksy ulegają fragmentacji. Z tego powodu administrator powinien regularnie planować operację **REINDEX**, która od nowa buduje strukturę drzewa indeksu, przywracając mu optymalny rozmiar i maksymalną wydajność. W systemach o wysokiej dostępności stosuje się przebudowę w trybie *CONCURRENTLY*, która nie przerywa pracy aplikacji.

2.3.6 Podsumowanie

Prawidłowa kontrola i konserwacja środowiska bazodanowego to wielowymiarowy proces, wymagający głębokiego zrozumienia zarówno architektury logicznej, jak i fizycznej serwera.

Stabilność i wydajność systemu zależy w równej mierze od prawidłowego zarządzania stanem usługi i połączeniami, dogłębnego planowania okien serwisowych, jak i proaktywnego zarządzania mechanizmem Vacuum i transakcjami. Wykorzystanie schematów do strukturyzacji danych oraz regularne dbanie o kondycję indeksów pozwala na zbudowanie środowiska, które będzie skalowalne, przewidywalne i odporne na awarie w trudnych, produkcyjnych warunkach.

Autorzy

1. Paweł Łoćwin
2. Paweł Łosowski

2.4 Monitorowanie i diagnostyka

Autorzy

1. Oskar Wrona
2. Kamil Lewandowski

3. Adam Tarkowski

2.4.1 Wstęp

Monitorowanie i diagnostyka baz danych obejmują zestaw działań, które pozwalają administratorowi lub zespołowi utrzymania sprawdzić, czy system działa poprawnie, bezpiecznie i wydajnie. W praktyce nie chodzi jedynie o obserwowanie pojedynczych parametrów serwera, ale o łączenie informacji pochodzących z wielu warstw: samej bazy danych, systemu operacyjnego, aplikacji oraz narzędzi zewnętrznych. Dzięki temu można szybciej wykrywać przeciążenia, błędy konfiguracji, problemy z zapytaniami SQL, nieprawidłowe uprawnienia oraz awarie sprzętowe lub sieciowe.

W systemach bazodanowych szczególnie ważne jest to, że wiele problemów nie jest od razu widocznych dla użytkownika końcowego. Przykładowo zapytania mogą stopniowo wykonywać się coraz wolniej, liczba aktywnych sesji może rosnąć, a logi mogą zawierać powtarzające się ostrzeżenia. Bez monitorowania takie sygnały są łatwe do przeoczenia. Diagnostyka pozwala natomiast przejść od samego wykrycia problemu do ustalenia jego przyczyny, np. przez analizę sesji, blokad, planów zapytań, dzienników błędów lub zużycia zasobów systemowych.

2.4.2 Sesje i użytkownicy

Jednym z podstawowych obszarów monitorowania bazy danych jest obserwowanie aktywnych sesji i użytkowników. Sesja oznacza połączenie użytkownika lub aplikacji z serwerem bazy danych. W ramach sesji wykonywane są zapytania, transakcje oraz operacje administracyjne. Analiza sesji pozwala sprawdzić między innymi, kto jest aktualnie połączony z bazą, z jakiego hosta pochodzi połączenie, jakie zapytanie jest wykonywane oraz czy sesja jest aktywna, bezczynna albo zablokowana.

W wielu systemach zarządzania bazami danych dostępne są specjalne widoki lub narzędzia administracyjne służące do obserwowania bieżącej aktywności. Na przykład PostgreSQL udostępnia mechanizmy monitorowania aktywności bazy danych oraz widoki statystyczne, takie jak `pg_stat_activity`, które pozwalają sprawdzać informacje o procesach serwera i wykonywanych zapytaniach¹. W MySQL podobną rolę może pełnić Performance Schema, które gromadzi dane diagnostyczne o pracy serwera i może być wykorzystywane do analizy wydajności².

Monitorowanie sesji jest istotne zarówno z punktu widzenia wydajności, jak i bezpieczeństwa. Zbyt duża liczba jednoczesnych połączeń może prowadzić do przeciążenia serwera, a długo działające transakcje mogą blokować inne operacje. Z kolei nietypowe połączenia, np. z nieznanymi adresów lub wykonywane poza normalnymi godzinami pracy, mogą wskazywać na błąd konfiguracji albo próbę nieautoryzowanego dostępu.

¹ PostgreSQL Documentation, *Monitoring Database Activity* oraz *The Cumulative Statistics System*, <https://www.postgresql.org/docs/current/monitoring.html>

² MySQL Documentation, *Using the Performance Schema to Diagnose Problems*, <https://dev.mysql.com/doc/refman/8.2/en/performance-schema-examples.html>

2.4.3 Śledzenie dostępu użytkowników do tabel

Kolejnym ważnym elementem jest kontrola tego, którzy użytkownicy uzyskują dostęp do określonych tabel i jakie operacje wykonują. W bazach danych przechowywane są często dane wrażliwe, dlatego sama informacja o tym, że użytkownik posiada konto w systemie, nie jest wystarczająca. Należy również wiedzieć, czy jego działania są zgodne z nadanymi uprawnieniami oraz z zasadami bezpieczeństwa obowiązującymi w organizacji.

Śledzenie dostępu może obejmować operacje odczytu danych, modyfikacji rekordów, usuwania danych, tworzenia nowych obiektów oraz zmian w uprawnieniach. W praktyce realizuje się to za pomocą mechanizmów audytu, dzienników zapytań, wyzwalaczy, narzędzi klasy Database Activity Monitoring lub funkcji wbudowanych w konkretny system bazodanowy. Przykładowo Oracle Database udostępnia mechanizmy audytu, a nowsze wersje zalecają korzystanie z unified auditing zamiast starszego audytu tradycyjnego³.

Takie śledzenie jest potrzebne nie tylko przy wykrywaniu incydentów bezpieczeństwa. Pomaga również w spełnianiu wymagań formalnych, analizie błędów użytkowników oraz odtwarzaniu przebiegu zdarzeń po awarii. Przykładowo, jeśli w tabeli pojawiły się nieprawidłowe dane, historia dostępu może pomóc ustalić, kiedy doszło do zmiany i która aplikacja lub który użytkownik ją wykonał.

2.4.4 Błędy dziennika, logi i raporty

Dzienniki zdarzeń, czyli logi, są jednym z najważniejszych źródeł informacji diagnostycznych. Serwer bazy danych może zapisywać w nich komunikaty o błędach, ostrzeżeniach, nieudanych logowaniach, problemach z zapytaniami, restartach, zmianach konfiguracji lub przekroczeniu określonych progów wydajnościowych. Analiza logów pozwala wykrywać problemy, które nie zawsze są widoczne w aplikacji klienckiej.

W zależności od systemu bazodanowego logi mogą mieć różną postać. PostgreSQL obsługuje różne metody logowania komunikatów serwera, między innymi `stderr`, `csvlog`, `jsonlog` oraz `syslog`; w systemie Windows możliwe jest także użycie dziennika zdarzeń⁴. MySQL udostępnia między innymi `general query log`, który może rejestrować połączenia i zapytania otrzymywane przez serwer, oraz `slow query log`, używany do identyfikowania zapytań wykonujących się zbyt długo⁵.

Raporty diagnostyczne są rozwinięciem surowych logów. Zamiast ręcznie analizować tysiące wpisów, administrator może korzystać z raportów podsumowujących liczbę błędów, najwolniejsze zapytania, częstotliwość nieudanych logowań lub zmiany obciążenia w czasie. Raporty są szczególnie przydatne w pracy zespołowej, ponieważ pozwalają dokumentować problemy i porównywać stan systemu przed oraz po wprowadzeniu zmian optymalizacyjnych.

³ Oracle Documentation, *AUDIT_TRAIL* oraz informacje o unified auditing, https://docs.oracle.com/en/database/oracle/oracle-database/21/refrn/AUDIT_TRAIL.html

⁴ PostgreSQL Documentation, *Error Reporting and Logging*, <https://www.postgresql.org/docs/current/runtime-config-logging.html>

⁵ MySQL Documentation, *The General Query Log* oraz *The Slow Query Log*, <https://dev.mysql.com/doc/en/query-log.html>

2.4.5 Monitorowanie na poziomie systemu operacyjnego

Baza danych działa na konkretnym systemie operacyjnym i korzysta z jego zasobów. Dlatego diagnostyka nie może ograniczać się wyłącznie do samego silnika bazy danych. Wydajność zapytań może zależeć od procesora, pamięci RAM, operacji wejścia/wyjścia na dysku, sieci, liczby procesów oraz ogólnego obciążenia systemu.

W systemach Linux często wykorzystuje się narzędzia takie jak `iostat`, `htop` oraz `vmstat`. Narzędzie `iostat` pozwala obserwować wykorzystanie procesora i urządzeń wejścia/wyjścia, co jest szczególnie ważne przy bazach danych intensywnie korzystających z dysku. `htop` umożliwia wygodne śledzenie procesów i zużycia zasobów w czasie rzeczywistym. `vmstat` pokazuje między innymi informacje o pamięci, procesach, wymianie danych z dyskiem oraz obciążeniu procesora.

W systemach Microsoft Windows podobną rolę pełnią Menedżer zadań oraz Monitor wydajności. Pozwalają one obserwować zużycie procesora, pamięci, dysków, sieci oraz usług działających w tle. W środowiskach produkcyjnych takie dane są często zbierane automatycznie i zestawiane z metrykami pochodzącymi z bazy danych. Dzięki temu można odróżnić problem wynikający z nieoptymalnego zapytania SQL od problemu spowodowanego np. przeciążeniem dysku lub brakiem pamięci.

2.4.6 Monitorowanie serwera

Ostatnim poziomem jest monitorowanie całego serwera lub infrastruktury, w której działa baza danych. Obejmuje ono dostępność usług, czas odpowiedzi, zużycie zasobów, stan dysków, wykorzystanie sieci, działanie kopii zapasowych oraz wysyłanie powiadomień w przypadku awarii. W tym celu stosuje się narzędzia takie jak Nagios, Grafana, Prometheus, Zabbix lub inne systemy monitoringu.

Nagios jest przykładem narzędzia używanego do monitorowania usług i hostów, sprawdzania ich stanu oraz informowania administratorów o problemach⁶. Grafana z kolei służy przede wszystkim do wizualizacji danych monitorujących w formie dashboardów. Panele Grafany mogą prezentować metryki w postaci wykresów, tabel i innych wizualizacji, a moduł alertów pozwala reagować na określone zdarzenia lub przekroczenie progów⁷.

Monitorowanie serwera jest szczególnie ważne w środowiskach, w których baza danych stanowi element większego systemu. Awaria bazy może wynikać nie tylko z błędu samego silnika, ale również z problemu sieciowego, pełnionego dysku, braku pamięci, błędnej konfiguracji systemu lub nieudanego wdrożenia aplikacji. Centralne monitorowanie pozwala zebrać te informacje w jednym miejscu i szybciej wskazać źródło problemu.

2.4.7 Znaczenie monitorowania i diagnostyki w bazach danych

Monitorowanie i diagnostyka są ważne, ponieważ wpływają na trzy kluczowe obszary: dostępność, wydajność i bezpieczeństwo danych. Dostępność oznacza, że baza danych jest osiągalna dla użytkowników i aplikacji. Wydajność oznacza, że zapytania wykonują się w akceptowalnym czasie. Bezpieczeństwo oznacza natomiast, że dostęp do danych jest kontrolowany, a podejrzane działania mogą zostać wykryte i przeanalizowane.

⁶ Nagios Open Source Documentation, <https://www.nagios.org/documentation/>

⁷ Grafana Documentation, *Dashboards overview* oraz *Grafana Alerting*, <https://grafana.com/docs/grafana/latest/fundamentals/dashboards-overview/>

W dobrze utrzymanym środowisku bazodanowym monitorowanie powinno działać stale, a nie dopiero wtedy, gdy pojawi się awaria. Regularne obserwowanie sesji, logów, dostępu do tabel, parametrów systemu operacyjnego i stanu serwera pozwala wykrywać problemy na wczesnym etapie. Diagnostyka pozwala natomiast ustalić przyczyny tych problemów i zaplanować odpowiednie działania, takie jak optymalizacja zapytań, zmiana konfiguracji, zwiększenie zasobów, poprawa uprawnień lub wdrożenie dodatkowych zabezpieczeń.

2.4.8 Podsumowanie

Monitorowanie i diagnostyka baz danych tworzą wielowarstwowy proces kontroli działania systemu. Na poziomie bazy danych analizuje się sesje, użytkowników, zapytania, blokady, dostęp do tabel oraz logi. Na poziomie systemu operacyjnego obserwuje się zużycie procesora, pamięci, dysku i sieci. Na poziomie serwera wykorzystuje się narzędzia monitorujące, które zbierają metryki, tworzą wykresy oraz wysyłają alerty. Dopiero połączenie tych perspektyw daje pełny obraz stanu systemu i pozwala skutecznie reagować na błędy, spadki wydajności oraz potencjalne incydenty bezpieczeństwa.

2.4.9 Źródła

2.5 Wydajność, skalowanie i replikacja

Autorzy

1. Olaf Chomicki
2. Konrad Machowski
3. Wiktor Wydrzyński

2.5.1 Wydajność, skalowanie i replikacja

Współczesne systemy bazodanowe muszą sprostać wymaganiom związanym z wykładniczym przyrostem wolumenu danych oraz stale rosnącą liczbą jednoczesnych użytkowników. W tym kontekście kluczowe staje się zrozumienie trzech fundamentalnych pojęć: wydajności, skalowania i replikacji. Wydajność odnosi się do szybkości przetwarzania pojedynczych transakcji i zapytań przy optymalnym wykorzystaniu dostępnych zasobów.

Gdy optymalizacja kodu przestaje wystarczać, konieczne staje się skalowanie – pionowe (zwiększanie mocy pojedynczego serwera) lub poziome (rozpraszanie obciążenia na wiele maszyn). Replikacja z kolei stanowi most łączący skalowanie odczytów z zapewnieniem wysokiej dostępności i odporności systemu na awarie sprzętowe.

2.5.2 Kontrola i buforowanie połączeń z bazą danych

Nawiązywanie nowego połączenia z bazą danych jest operacją niezwykle kosztowną procesorowo i czasowo, ponieważ wymaga uwierzytelnienia użytkownika, alokacji pamięci oraz ustanowienia sesji sieciowej. W systemach o dużym natężeniu ruchu brak kontroli nad liczbą połączeń prowadzi do szybkiego wyczerpania zasobów serwera.

Rozwiązaniem tego problemu jest buforowanie połączeń (Connection Pooling). Mechanizm ten polega na utrzymywaniu stałej puli otwartych, gotowych do użycia połączeń, które są wielokrotnie współdzielone przez aplikację. Narzędzia takie jak PgBouncer (dla PostgreSQL) czy wbudowane pule w serwerach aplikacji drastycznie zmniejszają narzut overhead, stabilizując czas reakcji bazy danych pod dużym obciążeniem.

Przykład konfiguracji PgBouncer (fragment pliku `pgbouncer.ini`):

```
[databases]
moja_baza = host=127.0.0.1 port=5432 dbname=produkcja

[pgbouncer]
listen_port = 6432
listen_addr = *
auth_type = md5
auth_file = /etc/pgbouncer/userlist.txt
pool_mode = transaction
max_client_conn = 1000
default_pool_size = 20
```

2.5.3 INDEX i CLUSTER

Indeksowanie to podstawowa technika optymalizacji zapytań o charakterze odczytowym. Instrukcja `INDEX` tworzy pomocniczą strukturę danych (najczęściej w formie B-drzewa), która pozwala silnikowi bazy danych na błyskawiczne odnalezienie żądanych wierszy bez konieczności kosztownego przeszukiwania całej tabeli (Full Table Scan).

Z kolei polecenie `CLUSTER` idzie o krok dalej – fizycznie reorganizuje strukturę danych na dysku w taki sposób, aby kolejność wierszy w tabeli dokładnie odpowiadała kolejności indeksu. Powoduje to, że powiązane rekordy są składowane obok siebie w tych samych blokach pamięci, co radykalnie przyspiesza zapytania zakresowe (Range Queries).

Przykład użycia poleceń w PostgreSQL:

```
-- 1. Tworzenie standardowego indeksu B-drzewa na kolumnie data_
--    zamowienia
CREATE INDEX idx_zamowienia_data ON zamowienia (data_zamowienia);

-- 2. Fizyczne przemieszczenie danych na dysku według kolejności_
--    indeksu
CLUSTER zamowienia USING idx_zamowienia_data;
```

2.5.4 Rola i zastosowanie replikacji

Replikacja polega na permanentnym kopiowaniu i synchronizowaniu danych z głównego węzła bazy danych (Primary/Master) na węzły pomocnicze (Replica/Slave). Jej rola w architekturze systemów IT jest wieloaspektowa. Po pierwsze, gwarantuje wysoką dostępność (High Availability) – w przypadku fizycznej awarii Mastera, jedna z replik może automatycznie przejąć jego funkcje (proces Failover).

Po drugie, umożliwia skalowanie odczytów (Read Scalability). Przekierowanie operacji modyfikujących (INSERT, UPDATE) do węzła głównego, a operacji odczytu oraz generowania ciężkich raportów na repliki pozwala na znaczne odciążenie głównego serwera.

2.5.5 Oprogramowanie i zaimplementowane mechanizmy replikacji

Mechanizmy replikacji mogą być realizowane na poziomie silnika bazy danych lub za pomocą zewnętrznego oprogramowania. Wyróżnia się dwa główne podejścia pod kątem transmisji danych: replikację opartą na logu (WAL - Write-Ahead Logging) oraz replikację logiczną. Ze względu na synchronizację, proces ten może przebiegać synchronicznie lub asynchronicznie.

Przykład konfiguracji replikacji logicznej (wbudowanej w PostgreSQL):

```
-- KROK 1: Uruchomienie na węźle głównym (Primary) - zdefiniowanie
-- publikacji
CREATE PUBLICATION pub_dane_sprzedazy FOR TABLE zamowienia, klienci;

-- KROK 2: Uruchomienie na węźle pomocniczym (Replica) - subskrypcja
-- danych
CREATE SUBSCRIPTION sub_dane_sprzedazy
CONNECTION 'host=192.168.1.50 port=5432 dbname=produkcja user=repl_
-- user password=secret'
PUBLICATION pub_dane_sprzedazy;
```

2.5.6 Limity systemu oraz ograniczanie dostępu użytkowników

Bezpieczeństwo i stabilność bazy danych wymagają rygorystycznego zarządzania limitami systemowymi oraz uprawnieniami. Zbyt duża swoboda przyznana użytkownikom lub procesom aplikacyjnym może doprowadzić do celowego bądź przypadkowego unieruchomienia systemu.

Przykład nakładania restrykcji i limitów zasobów w PostgreSQL:

```
-- 1. Ograniczenie maksymalnej liczby jednoczesnych połączeń dla danej
-- roli
ALTER ROLE rola_analitik CONNECTION LIMIT 5;

-- 2. Ustawienie maksymalnego czasu wykonywania pojedynczego zapytania
-- (np. 30 sekund)
ALTER ROLE rola_analitik SET statement_timeout = '30s';

-- 3. Nadanie uprawnień wyłącznie do odczytu danych w wybranym
-- schemacie
REVOKE ALL ON ALL TABLES IN SCHEMA public FROM rola_analitik;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO rola_analitik;
```

2.5.7 Testy wydajności sprzętu na poziomie systemu operacyjnego

Przed wdrożeniem produkcyjnym bazy danych kluczowe jest przeprowadzenie niezależnych testów wydajnościowych podzespołów sprzętowych (Benchmarków) bezpośrednio na poziomie systemu operacyjnego, aby wykluczyć błędy konfiguracyjne środowiska.

Przykłady poleceń diagnostycznych i testowych w systemie Linux:

```
# 1. Test wydajności procesora (CPU) za pomocą sysbench
sysbench cpu --cpu-max-prime=20000 run

# 2. Test przepustowości i alokacji pamięci RAM
sysbench memory --memory-block-size=1M --memory-total-size=10G run

# 3. Zaawansowany test wydajności dysku (IOPS i opóźnienia) narzędziem
→ fio
# Symulacja losowego odczytu/zapisu (Random R/W) specyficznego dla baz
→ danych
fio --name=test_bazy --ioengine=libaio --rw=randrw --bs=4k \
    --size=2G --numjobs=2 --runtime=30 --time_based \
    --filename=/var/lib/postgresql/test_io.file --group_reporting
```

2.6 Partycjonowanie danych

Autorzy

1. Michał Bystrzak
2. Damian Dominiak

2.6.1 Wstęp i istota problemu

W dobie systemów informatycznych przetwarzających gigabajty lub terabajty informacji (tzw. VLDB - Very Large Databases), tradycyjne podejście do składowania danych staje się wysoce niewydajne. Kiedy pojedyncza tabela rozrasta się do setek milionów rekordów, fizyczne ograniczenia sprzętowe oraz narzuty na zarządzanie indeksami powodują drastyczny spadek wydajności. Partycjonowanie to zaawansowana technika architektoniczna, polegająca na logicznym podziale jednej, ogromnej tabeli na mniejsze, odseparowane fizycznie fragmenty nazywane partycjami. Z perspektywy aplikacji docelowej oraz zapytań SQL, tabela wciąż prezentuje się i zachowuje jak spójna, pojedyncza relacja.

2.6.2 Cele stosowania partycjonowania danych

Implementacja mechanizmu partycjonowania w systemach relacyjnych jest podyktowana chęcią rozwiązania konkretnych problemów wydajnościowych i administracyjnych. Główne cele to:

- **Zwiększenie wydajności zapytań (Query Performance):** Fundamentalnym celem jest eliminacja konieczności skanowania całej tabeli (Full Table Scan). Dzięki mechanizmowi *Partition Pruning* (odcinanie partycji), planista zapytań analizuje warunki zdefiniowane w

klauzuli WHERE i kieruje odczyt wyłącznie do tych fizycznych plików, które mogą zawierać poszukiwane dane.

- **Optymalizacja procesów utrzymaniowych (Maintenance):** Olbrzymie tabele są trudne w utrzymaniu. Operacje takie jak VACUUM, REINDEX czy odtwarzanie kopii zapasowych mogą blokować zasoby przez wiele godzin. Partycjonowanie pozwala na wykonywanie tych operacji selektywnie, tylko na tych partycjach, które uległy modyfikacji.
- **Zarządzanie cyklem życia danych (Data Lifecycle Management):** W przypadku danych historycznych (np. logów), najszybszą metodą ich usunięcia jest operacja DROP TABLE na starej partycji. Jest to operacja na metadanych, trwająca ułamki sekund, w przeciwieństwie do standardowego DELETE, które w architekturze MVCC (Multi-Version Concurrency Control) generuje ogromne ilości „martwych krotek” (dead tuples) i obciąża dziennik transakcji (WAL).

2.6.3 Zastosowanie partycjonowania

Technika ta nie jest uniwersalnym rozwiązaniem dla każdej bazy danych, lecz sprawdza się w specyficznych, wymagających scenariuszach:

- **Hurtownie danych (OLAP):** Systemy analityczne gromadzące dane historyczne. Partycjonowanie pozwala tam na błyskawiczne agregowanie danych (np. sumowanie przychodów z konkretnego miesiąca) bez wczytywania do pamięci RAM danych z całej dekady.
- **Systemy transakcyjne (OLTP) o wysokim przyroście:** Aplikacje rejestrujące logi systemowe, odczyty z czujników IoT, dane telemetryczne czy rejestry transakcji finansowych (Time-Series Data), gdzie każdego dnia przybywają miliony nowych wierszy.
- **Implementacja Storage Tiering:** Możliwość fizycznego rozdzielenia danych na różne nośniki. Najnowsze i najczęściej odpytywane partycje (Hot Data) umieszcza się na ultraszybkich macierzach NVMe, podczas gdy starsze, rzadko używane partycje (Cold Data) przenosi się na tańsze wolumeny dyskowe (HDD), co znacząco optymalizuje koszty infrastruktury.

2.6.4 Zalety i wady partycjonowania

Jak każda technika optymalizacyjna, partycjonowanie stanowi kompromis inżynierski (Trade-off).

Zalety:

- Spektakularny wzrost wydajności operacji odczytu, o ile zapytanie wykorzystuje klucz partycjonowania.
- Optymalne wykorzystanie pamięci operacyjnej (RAM) – indeksy dla poszczególnych partycji są znacznie mniejsze i z większym prawdopodobieństwem zmieszczą się w całości w pamięci podręcznej (Cache).
- Możliwość bezinwazyjnego odłączania i dołączania gigabajtów danych niemal w czasie rzeczywistym.

Wady:

- Zwiększony narzut (overhead) na planistę zapytań. Przy bardzo dużej liczbie partycji (np. tysiące fragmentów), czas samego planowania zapytania przez silnik bazy może być dłuższy niż czas jego wykonania.
- Ograniczenia strukturalne: w wielu systemach DBMS (w tym w PostgreSQL) globalne klucze główne (Primary Keys) i ograniczenia unikalności (UNIQUE) muszą z założenia zawierać w sobie kolumnę będącą kluczem partycjonowania, co bywa trudne do pogodzenia z logiką biznesową aplikacji.
- Spadek wydajności w przypadku zapytań omijających klucz partycjonowania – silnik musi wówczas odpytać każdą fizyczną partycję z osobna.

2.6.5 Implementacja partycjonowania w PostgreSQL

Począwszy od wersji 10, PostgreSQL oferuje natywne wsparcie dla partycjonowania deklaratywnego. Eliminuje to konieczność ręcznego pisania skomplikowanych wyzwalaczy czy reguł (rules). Silnik obsługuje trzy główne strategie przydziału danych:

- **Zakresowe (RANGE):** Dane są dzielone na podstawie ciągłych przedziałów wartości. Najczęściej wykorzystywanym kluczem są typy daty i czasu (np. jedna partycja dla 2025-01-01 do 2025-01-31) lub sekwencje numeryczne.
- **Listowe (LIST):** Dane są dystrybuowane na podstawie konkretnych, dyskretnych wartości zdefiniowanych w tablicy. Metoda ta idealnie sprawdza się w przypadku podziału na kategorie biznesowe, takie jak „Status_Zlecenia” (osobna partycja dla wartości Zakończony, W trakcie) lub regiony geograficzne.
- **Haszujące (HASH):** Wiersze są rozdzielane na zadaną liczbę partycji (modułów) z wykorzystaniem zaawansowanej, matematycznej funkcji skrótu. Strategia ta nie służy przyspieszaniu zapytań zakresowych, lecz idealnie równomiernemu rozłożeniu ciężaru I/O podczas masowych operacji wstawiania (INSERT) w systemach wielodyskowych.

2.6.6 Ograniczenia i obejście w środowisku SQLite

W przeciwieństwie do potężnych, serwerowych silników RDBMS takich jak PostgreSQL, lekka i plikowa biblioteka SQLite **nie posiada** natywnego mechanizmu partycjonowania deklaratywnego. Wynika to z jej osadzonej (embedded) architektury.

Niemniej jednak, efekt partycjonowania w SQLite można emulować za pomocą programistycznego podejścia warstwowego: 1. **Podział fizyczny:** Utworzenie wielu mniejszych, niezależnych tabel (np. zlecenia_2025, zlecenia_2026). 2. **Warstwa logiczna:** Stworzenie wspólnego widoku (VIEW), który scala wszystkie te tabele z wykorzystaniem operatora UNION ALL. Umożliwia to wykonywanie zapytań SELECT na jednej strukturze. 3. **Zarządzanie modyfikacjami:** Obsługa instrukcji DML (INSERT, UPDATE, DELETE) wymaga zdefiniowania na widoku zestawu wyzwalaczy (TRIGGERS INSTEAD OF). Wyzwalacze te muszą przechwytywać dane w locie, analizować klucz (np. datę) i dynamicznie przekierowywać operację do odpowiedniej fizycznej pod-tabeli.

Choć to rozwiązanie jest funkcjonalne, należy pamiętać, że w SQLite każdorazowe uruchamianie wyzwalaczy generuje zauważalny narzut wydajnościowy w porównaniu do natywnego mechanizmu w PostgreSQL.

2.7 Bezpieczeństwo

Autor

Mateusz

2.7.1 Wstęp teoretyczny

Bezpieczeństwo danych jest jednym z kluczowych aspektów zarządzania systemami baz danych. Obejmuje ono ochronę przed nieuprawnionym dostępem, złośliwą modyfikacją lub utratą informacji, a także zapewnienie poufności, integralności i ciągłej dostępności usług. W zależności od wybranej architektury systemu, mechanizmy bezpieczeństwa mogą być wbudowane bezpośrednio w silnik bazy danych lub w całości delegowane do systemu operacyjnego oraz aplikacji klienckiej.

W niniejszym podrozdziale dokonano szczegółowej analizy porównawczej mechanizmów bezpieczeństwa zaimplementowanych w sieciowym systemie **PostgreSQL** oraz wbudowanym silniku bazodanowym **SQLite**.

2.7.2 Uwierzytelnianie i Kontrola Dostępu

PostgreSQL

PostgreSQL oferuje wysoce elastyczny podsystem uwierzytelniania, którego konfiguracja opiera się na pliku `pg_hba.conf` (Host-Based Authentication). Pozwala on na precyzyjne określenie, jaki użytkownik, z jakiego adresu IP i do jakiej bazy danych może się zalogować. System wspiera liczne zaawansowane metody weryfikacji tożsamości:

- **SCRAM-SHA-256**: Nowoczesny i bezpieczny domyślny mechanizm oparty na protokole wyzwanie-odpowiedź, uniemożliwiający podsłuchanie haseł.
- **Uwierzytelnianie zewnętrzne**: Integracja z korporacyjnymi systemami zarządzania tożsamością, takimi jak LDAP, Kerberos, GSSAPI, PAM czy RADIUS.
- **Certyfikaty SSL**: Możliwość uwierzytelniania klientów wyłącznie na podstawie ważnych certyfikatów kryptograficznych x509.

SQLite

W przeciwieństwie do systemów typu klient-serwer, SQLite nie posiada wbudowanego mechanizmu uwierzytelniania użytkowników. Wynika to bezpośrednio z jego architektury, gdzie baza danych jest po prostu pojedynczym plikiem na dysku.

- Kontrola dostępu do danych jest całkowicie zależna od uprawnień systemu operacyjnego.
- Każdy proces lub użytkownik systemu operacyjnego, który posiada uprawnienia systemowe do odczytu i zapisu pliku bazy danych, ma automatycznie pełny dostęp do wszystkich danych i struktur w niej zawartych.
- Istnieje oficjalne, opcjonalne rozszerzenie SQLite User Authentication, jednak nie jest ono częścią standardowej dystrybucji i zaprzecza podstawowej idei prostoty tego silnika.

2.7.3 Autoryzacja i Uprawnienia

PostgreSQL

Autoryzacja w PostgreSQL realizowana jest poprzez model kontroli dostępu oparty na rolach (RBAC – *Role-Based Access Control*). Za pomocą instrukcji `GRANT` i `REVOKE` administratorzy mogą szczegółowo definiować prawa:

- **Granularność uprawnień:** Uprawnienia mogą być nadawane na poziomie całych baz danych, schematów, tabel, widoków, a także konkretnych kolumn.
- **Row-Level Security (RLS):** Zaawansowany mechanizm polityk bezpieczeństwa, który filtruje zwracane wiersze w zależności od tożsamości zalogowanego użytkownika.

SQLite

Standardowy silnik SQLite nie obsługuje pojęcia ról ani wewnętrznych uprawnień.

- Po pomyślnym otwarciu pliku bazy danych, programista lub użytkownik może wykonać każdą operację (`SELECT`, `INSERT`, `DROP TABLE`).
- Jedyną formą ograniczenia uprawnień jest otwarcie bazy danych przez aplikację w trybie tylko do odczytu, użycie flagi wewnętrznej `SQLITE_OPEN_READONLY` podczas inicjalizacji połączenia w kodzie źródłowym.

2.7.4 Szyfrowanie Danych

PostgreSQL

PostgreSQL zapewnia ochronę danych zarówno w trakcie ich przesyłania, jak i podczas przechowywania na nośnikach:

- **Szyfrowanie w locie (Data-in-Transit):** Pełne i natywne wsparcie dla protokołu SSL/TLS, co zapobiega podsłuchiowaniu ruchu sieciowego między aplikacją a serwerem bazy danych.
- **Szyfrowanie w spoczynku (Data-at-Rest):** Standardowa wersja PostgreSQL nie posiada wbudowanego mechanizmu przezroczystego szyfrowania bazy danych (TDE). Bezpieczeństwo na dysku realizuje się za pomocą narzędzi systemowych lub poprzez wbudowane rozszerzenie `pgcrypto`, pozwalające na szyfrowanie konkretnych wartości w kolumnach za pomocą algorytmu AES.

SQLite

Ponieważ cała baza danych SQLite mieści się w jednym pliku, fizyczne skopiowanie tego pliku oznacza kradzież wszystkich danych. Szyfrowanie staje się więc kluczowe:

- **Brak szyfrowania w standardzie:** Domyślnie pliki bazy danych są zapisywane jawnym tekstem.
- **Szyfrowanie za pomocą rozszerzeń:** Aby zabezpieczyć bazę, stosuje się zewnętrzne biblioteki modyfikujące warstwę VFS (Virtual File System). Najpopularniejszym, otwartoźródłowym rozwiązaniem jest **SQLCipher** szyfrujący plik algorytmem AES-256 oraz oficjalne, komercyjne rozszerzenie **SEE** (*SQLite Encryption Extension*).

2.7.5 Audyt i Logowanie

PostgreSQL

System udostępnia zaawansowane mechanizmy śledzenia i rejestrowania operacji. Konfiguracja pozwala na logowanie błędów, prób uwierzytelnienia, a także pełnych zapytań SQL. W środowiskach wymagających rygorystycznego audytu stosuje się rozszerzenie **pgaudit**, które tworzy szczegółowe dzienniki zdarzeń zawierające informacje o tym, kto, kiedy i jakie dokładnie dane modyfikował lub przeglądał.

SQLite

SQLite nie generuje własnych plików dziennika zdarzeń przeznaczonych do audytu bezpieczeństwa. Śledzenie aktywności musi zostać zaimplementowane w kodzie aplikacji klienckiej przy użyciu funkcji API udostępnianych przez bibliotekę SQLite, takich jak mechanizmy przechwytyjące zapytania: `sqlite3_trace_v2()` lub filtry autoryzacyjne `sqlite3_set_authorizer()`.

2.7.6 Podsumowanie

Podsumowując, podejście do bezpieczeństwa w systemach DBMS zależy w głównej mierze od ich założeń architektonicznych. PostgreSQL, jako zaawansowany system typu klient-serwer, oferuje kompleksowy i scentralizowany zestaw narzędzi bezpieczeństwa na poziomie silnika. Dzięki separacji procesów, zaawansowanej kontroli dostępu oraz natywnej obsłudze szyfrowania sieciowego, stanowi on optymalne rozwiązanie dla środowisk wielodostępnych i rozproszonych.

Z kolei SQLite jest minimalny i prosty. Jako bezserwerowy silnik osadzony, w całości deleguje zadania związane z uwierzytelnianiem i autoryzacją użytkowników do systemu operacyjnego lub samej aplikacji klienckiej. Bezpieczeństwo danych sprowadza się tu do ochrony fizycznego pliku bazy danych na dysku, a wdrożenie bardziej zaawansowanych mechanizmów, takich jak kryptograficzna ochrona danych w spoczynku, wymaga implementacji zewnętrznych nakładek programistycznych.

2.8 Kopie zapasowe i odzyskiwanie danych

Autorzy: Norbert Antonovitch, Michał Bednarczyk, Jan Balazs de Borbatviz

2.8.1 Wstęp

Kopie zapasowe w PostgreSQL można podzielić na dwa główne nurty: kopie logiczne wykonywane narzędziami `pg_dump` i `pg_dumpall` oraz kopie fizyczne całego klastra wykonywane m.in. przez `pg_basebackup` wraz z archiwizacją WAL dla odzyskiwania punkowego PITR (Point-in-Time Recovery). Kopie logiczne są wygodne do odtwarzania pojedynczych obiektów i pojedynczych baz, natomiast kopie fizyczne są podstawą pełnego odtworzenia instancji, tablespaces i odzyskiwania po awariach.

2.8.2 Tworzenie kopii zapasowej całego systemu PostgreSQL - mechanizmy wbudowane

Najprostszym wbudowanym sposobem wykonania kopii całego klastra jest `pg_dumpall`, które eksportuje wszystkie bazy w klastrze oraz obiekty globalne wspólne dla całej instancji, takie jak role i przestrzenie tabel. Taki zrzut ma postać tekstowego skryptu SQL i odtwarza się go zwykle przez `psql`, ale metoda ta jest logiczna, więc nie daje możliwości odzyskiwania punktowego przez WAL.

Drugim, ważniejszym z punktu widzenia odporności na awarie mechanizmem jest `pg_basebackup`, które tworzy fizyczną kopię działającego klastra bez blokowania normalnej pracy klientów. Narzędzie to wykonuje kopię całego klastra, a nie pojedynczych baz czy tabel, może pracować w formacie katalogowym lub `tar` i może dołączać wymagane pliki WAL metodą `stream`, `fetch` albo `none`. Dokumentacja podkreśla też, że taka kopia może służyć zarówno do PITR, jak i do zbudowania serwera standby.

Aby pełna kopia fizyczna była użyteczna w scenariuszu odtwarzania do wybranego momentu, należy włączyć archiwizację WAL przez ustawienie co najmniej `wal_level = replica`, `archive_mode = on` oraz poprawnego `archive_command` albo `archive_library`. PostgreSQL zapisuje każdą zmianę w logu WAL, a po odtworzeniu kopii bazowej można odtworzyć kolejne segmenty WAL, aby dojść do stanu bieżącego lub do wskazanego punktu w czasie.

Przykładowe polecenia:

```
# logiczna kopia całego klastra
pg_dumpall -U postgres --clean --file=cluster.sql

# fizyczna kopia całego klastra z dołączeniem WAL
pg_basebackup -h dbserver -D /backup/base -Fp -X stream -P
```

W praktyce do dokumentacji laboratoryjnej warto zaznaczyć, że `pg_dumpall` lepiej nadaje się do migracji i prostych kopii administracyjnych, a `pg_basebackup` z archiwizacją WAL do scenariuszy disaster recovery.

2.8.3 Tworzenie kopii zapasowych poszczególnych baz danych - mechanizmy wbudowane

Do wykonania kopii pojedynczej bazy służy `pg_dump`, które tworzy spójny zrzut nawet wtedy, gdy baza jest równocześnie używana przez innych użytkowników, i nie blokuje zwykłych operacji odczytu ani zapisu. Narzędzie obsługuje format tekstowy, `custom`, `directory` oraz `tar`, przy czym formaty archiwalne współpracują z `pg_restore` i pozwalają na selektywne odtwarzanie obiektów.

Istotne jest to, że `pg_dump` wykonuje kopię tylko jednej bazy danych. Format `custom` (`-Fc`) i `directory` (`-Fd`) są najbardziej elastyczne, bo pozwalają wybierać odtwarzane elementy, zmieniać kolejność odtwarzania, a w części scenariuszy także korzystać z odtwarzania równoległego.

Przykładowe polecenia:

```
# kopia pojedynczej bazy w formacie custom
pg_dump -U postgres -d labdb -Fc -f /backup/labdb.dump

# kopia pojedynczej bazy jako skrypt SQL
pg_dump -U postgres -d labdb -Fp -f /backup/labdb.sql

# odtworzenie archiwum custom
pg_restore -d labdb_restored /backup/labdb.dump
```

Jeżeli celem jest ochrona wybranych schematów lub tabel, `pg_dump` pozwala zawęzić zakres eksportu przez opcje takie jak `-n` dla schematu i `-t` dla tabeli. Trzeba jednak zaznaczyć, że wybiórczy zrzut nie gwarantuje kompletności zależności wszystkich obiektów w nowym, pustym środowisku, jeśli pominięto elementy, od których zależy odtwarzany obiekt.

2.8.4 Odzyskiwanie usuniętych lub uszkodzonych danych

Sposób odzyskiwania zależy od tego, czy problem dotyczy pojedynczego obiektu logicznego, całej bazy, przestrzeni tabel, czy fizycznego uszkodzenia danych. PostgreSQL rozróżnia w praktyce odzyskiwanie logiczne z plików dump oraz odzyskiwanie fizyczne z kopii bazowej i archiwum WAL.

Odtwarzanie tabel i innych obiektów logicznych

Jeżeli wcześniej wykonano kopię `pg_dump` w formacie archiwalnym, można odtworzyć pojedynczą tabelę lub inny obiekt selektywnie przy pomocy `pg_restore`, bez przywracania całej bazy. To jest najprostszy wariant odzyskania przypadkowo usuniętej tabeli, o ile odpowiedni obiekt znajdował się w logicznej kopii zapasowej.

Przykład:

```
pg_restore -d labdb -t public.wyniki /backup/labdb.dump
```

Odtwarzanie usuniętej bazy danych

Usuniętą bazę można najłatwiej odtworzyć z wcześniejszego zrzutu `pg_dump` tej konkretnej bazy albo z `pg_dumpall`, jeżeli wykonywano kopię całego klastra. W przypadku wymogu odtworzenia dokładnie do chwili sprzed usunięcia, potrzebna jest kopia fizyczna oraz archiwum WAL, ponieważ same zrzuty logiczne nie są częścią rozwiązania continuous archiving i nie wspierają PITR.

Odtwarzanie przestrzeni tabel i pełnego klastra

Przestrzenie tabel są obiektami globalnymi klastra, dlatego ich definicje są uwzględniane przez `pg_dumpall`, a przy kopiach fizycznych `pg_basebackup` zachowuje także ich strukturę i mapowanie. Dokumentacja `pg_basebackup` wskazuje, że przy pracy z tablespaces można użyć mapowania `--tablespace-mapping`, a przy odtwarzaniu trzeba zweryfikować poprawność dowiązań symbolicznych i lokalizacji danych.

Przy fizycznym uszkodzeniu danych, katalogu PGDATA albo tablespaces standardowa procedura obejmuje odtworzenie kopii bazowej, usunięcie starych plików danych, odtworzenie katalogów danych i tablespaces, skonfigurowanie `restore_command`, utworzenie pliku `recovery.signal`, a następnie ponowne uruchomienie serwera w trybie `recovery`. PostgreSQL podczas odtwarzania odczytuje wymagane segmenty WAL i może dojść do stanu bieżącego albo zatrzymać się na wybranym punkcie odzyskiwania.

PITR i odzyskiwanie stanu sprzed błędu

Najważniejszym mechanizmem odzyskiwania po usunięciu danych operacyjnych jest PITR, czyli przywrócenie systemu do chwili sprzed błędnej operacji, na przykład `DROP TABLE` lub `DROP DATABASE`. W takim scenariuszu odtwarza się kopię bazową, konfiguruje `restore_command`, a następnie ustawia cel odzyskiwania, np. przez czas, nazwany punkt przywracania lub identyfikator transakcji.

To rozwiązanie ma jednak ważne ograniczenie: nie służy do odzyskania pojedynczej tabeli „w miejscu” bez wpływu na resztę systemu, lecz do odtworzenia całego klastra do wcześniejszego stanu. Dlatego w praktyce laboratoryjnej często łączy się oba podejścia: regularne `pg_dump` dla obiektów logicznych i `pg_basebackup` z archiwizacją WAL dla pełnego `recovery`.

2.8.5 Dedykowane oprogramowanie i skrypty zewnętrzne do automatyzacji

Wbudowane narzędzia PostgreSQL są wystarczające do ręcznego wykonywania kopii, ale w środowisku produkcyjnym często wykorzystuje się oprogramowanie automatyzujące harmonogramy, retencję, walidację i odtwarzanie. Dwa najczęściej wskazywane rozwiązania to Barman i pgBackRest, przy czym dostępna dokumentacja Barman bezpośrednio opisuje obsługę pełnych i przyrostowych kopii, archiwizacji WAL oraz automatycznego uruchamiania backupów.

Barman wykonuje kopie całego serwera PostgreSQL i wymaga poprawnej archiwizacji WAL, oferując różne metody kopii, w tym streaming przez `pg_basebackup`, `rsync` oraz backupy snapshotowe w chmurze. Dokumentacja podaje też wsparcie dla backupów przyrostowych, limitowania pasma, kompresji i pracy z harmonogramem przez zewnętrzne mechanizmy systemowe, np. `cron`.

Przykładowe zastosowania narzędzi zewnętrznych:

- **Barman:** centralne repozytorium backupów, archiwizacja WAL, backupy pełne i przyrostowe, odzyskiwanie całych instancji.
- **pgBackRest:** popularne narzędzie do wydajnych backupów i retencji, często używane w środowiskach HA oraz większych instalacjach PostgreSQL.
- **Własne skrypty Bash/PowerShell:** wywołanie `pg_dump`, `pg_dumpall` lub `pg_basebackup`, kompresja plików, rotacja katalogów, logowanie i wysyłka wyników przez e-mail lub do systemu monitoringu.

Przykładowy prosty skrypt automatyzujący kopię jednej bazy:

```
#!/bin/bash
DATA=$(date +%F_%H-%M)
```

(ciąg dalszy na następnej stronie)

(kontynuacja poprzedniej strony)

```
KATALOG=/backup/postgres
```

```
BAZA=labdb
```

```
mkdir -p "$KATALOG"
```

```
pg_dump -U postgres -d "$BAZA" -Fc -f "$KATALOG/${BAZA}_${DATA}.dump"
```

```
find "$KATALOG" -type f -name "${BAZA}*.dump" -mtime +7 -delete
```

Taki skrypt jest prosty, ale nie zapewnia pełnego disaster recovery, bo nie obejmuje archiwizacji WAL i nie chroni całego klastra. Z tego powodu dla systemów krytycznych bardziej właściwe są narzędzia klasy Barman lub pgBackRest, które porządkują pełny proces backupu, retencji, testów i odtwarzania.

2.8.6 Podsumowanie

W rozdziale warto wyraźnie rozróżnić kopie logiczne i fizyczne, bo odpowiadają one na inne potrzeby eksploatacyjne. `pg_dump` i `pg_dumpall` są najlepsze do prostych eksportów i selektywnego odtwarzania, natomiast `pg_basebackup` z archiwizacją WAL jest podstawą odzyskiwania po poważnej awarii i realizacji PITR.

Planowanie baz danych i tworzenie dokumentacji

Autor: Mateusz Gałęcki

3.1 Wybór zagadnienia i opis procesów

Do stworzenia wybrałem baze danych obsługującą system kina.

Opis głównych procesów:

- **Zarządzanie katalogiem filmów:** Wprowadzanie danych filmowych
- **Zarządzanie infrastrukturą:** Dane o salach kinowych ich pojemności i sprzedaży biletów
- **Planowanie repertuaru:** Przypisanie filmów dla konkretnych sal

3.2 Prototypowe dane

Prototypowe pliki w formatach CSV i JSON.

Format CSV:

```
tytul_filmu,rok_produkcji,czas_trwania,nazwa_sali,pojemnosc,czas_
→rozpoczecia
Matrix,1999,136,Sala 1,120,2026-06-25 18:00:00
```

Format JSON:

```
{
  "tytul": "Matrix",
```

(ciąg dalszy na następnej stronie)

(kontynuacja poprzedniej strony)

```

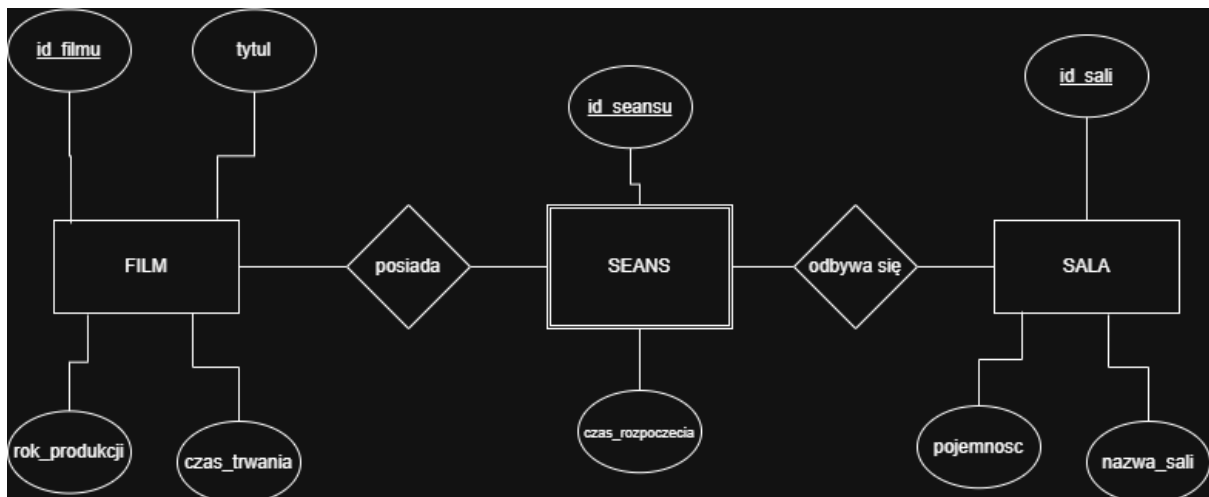
"rok_produkcji": 1999,
"czas_trwania": 136,
"nazwa_sali": "Sala 1",
"pojemnosc": 120,
"czas_roz poczenia": "2026-06-25 18:00:00"
}

```

3.3 Model konceptualny

Podzieliłem to na następujące elementy:

- **Encje silne:** Film, Sala (ich istnienie jest niezależne od innych)
- **Encje słabe:** Seans (jej istnienie jest uzależnione od innych encji)
- **Związki niepoprawne:** pozbyłem się związków wiele-do-wielu pomiędzy filmem a salą.



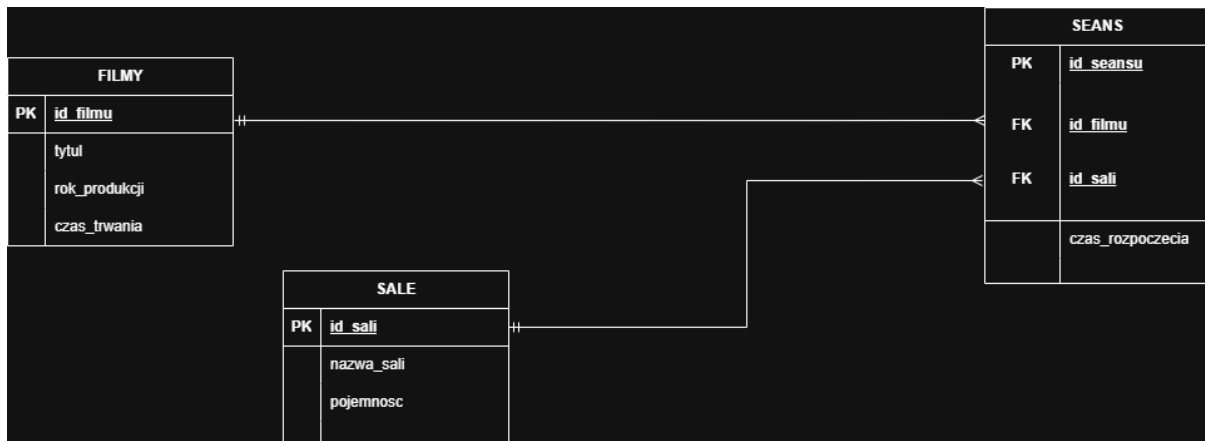
3.4 Model logiczny i normalizacja

Dane z prototypu poddałem normalizacji osiągając 3 postać normalną (3NF).

1. **1NF:** każda informacja znajduje się w osobnej kolumnie
2. **2NF i 3NF:** Usunąłem zależności częściowe i przechodnie.

Związki między znormalizowanymi tabelami mają postać 1:N

- Jeden FILM może posiadać wiele SEANSÓW.
- Jedna SALA może posiadać wiele SEANSÓW.



3.5 Model fizyczny bazy danych

Model bazy danych z podziałem na PostgreSQL i SQLite

PostgreSQL

Tabela 1: Model fizyczny bazy danych (PostgreSQL)

Tabela	Kolumna	Typ danych	Klucz / Więzy
filmy	id_filmu	SERIAL	PK
filmy	tytul	VARCHAR(255)	NOT NULL
filmy	rok_produkcji	INT	
filmy	czas_trwania	INT	NOT NULL
sale	id_sali	SERIAL	PK
sale	nazwa_sali	VARCHAR(50)	NOT NULL
sale	pojemnosc	INT	NOT NULL
seanse	id_seansu	SERIAL	PK
seanse	id_filmu	INT	FK (filmy.id_filmu)
seanse	id_sali	INT	FK (sale.id_sali)
seanse	czas_rozpoczecia	TIMESTAMP	NOT NULL

SQLite

Tabela 2: Model fizyczny bazy danych (SQLite)

Tabela	Kolumna	Typ danych	Klucz / Więzy
filmy	id_filmu	INTEGER	PK AUTOINCREMENT
filmy	tytul	TEXT	NOT NULL
filmy	rok_produkcji	INTEGER	
filmy	czas_trwania	INTEGER	NOT NULL
sale	id_sali	INTEGER	PK AUTOINCREMENT
sale	nazwa_sali	TEXT	NOT NULL
sale	pojemnosc	INTEGER	NOT NULL
seanse	id_seansu	INTEGER	PK AUTOINCREMENT
seanse	id_filmu	INTEGER	FK (filmy.id_filmu)
seanse	id_sali	INTEGER	FK (sale.id_sali)
seanse	czas_rozpoczecia	DATETIME	NOT NULL

Definiowanie bazy danych i wprowadzanie danych

Autor: Mateusz Gałeczki

4.1 Definiowanie struktury bazy danych

Na podstawie modeli fizycznych z rozdziału 3 przygotowałem skrypt pozwalający na utworzenie struktury tabel.

PostgreSQL:

```
CREATE TABLE filmy (  
    id_filmu SERIAL PRIMARY KEY,  
    tytul VARCHAR(255) NOT NULL,  
    rok_produkcji INT,  
    czas_trwania INT NOT NULL  
);  
  
CREATE TABLE sale (  
    id_sali SERIAL PRIMARY KEY,  
    nazwa_sali VARCHAR(50) NOT NULL,  
    pojemnosc INT NOT NULL  
);  
  
CREATE TABLE seanse (  
    id_seansu SERIAL PRIMARY KEY,  
    id_filmu INT REFERENCES filmy(id_filmu),  
    id_sali INT REFERENCES sale(id_sali),  
    czas_rozpoczecia TIMESTAMP NOT NULL  
);
```

SQLite:

```
CREATE TABLE filmy (  
    id_filmu INTEGER PRIMARY KEY AUTOINCREMENT,  
    tytul TEXT NOT NULL,  
    rok_produkcji INTEGER,  
    czas_trwania INTEGER NOT NULL  
);  
  
CREATE TABLE sale (  
    id_sali INTEGER PRIMARY KEY AUTOINCREMENT,  
    nazwa_sali TEXT NOT NULL,  
    pojemnosc INTEGER NOT NULL  
);  
  
CREATE TABLE seanse (  
    id_seansu INTEGER PRIMARY KEY AUTOINCREMENT,  
    id_filmu INTEGER REFERENCES filmy(id_filmu),  
    id_sali INTEGER REFERENCES sale(id_sali),  
    czas_roz poczeczcia DATETIME NOT NULL  
);
```

4.2 Wybór mechanizmów wsadowego wprowadzania danych

Gotowe tabele musialem uzupełnić danymi. Brałem pod uwagę następujące mechanizmy wsadowego wprowadzania danych:

- **PostgreSQL:** Środowisko to oferuje bardzo dobry import danych z plików CSV za pomocą polecenia „COPY”. Jest to najszybsza metoda ale wymaga ona kontroli na formacie pliku wsadowego. Drugą opcją jest funkcja „INSERT” która pozwala na wprowadzenie wielu wierszy jednym zapytaniem.
- **SQLite:** W SQLite najbardziej znaną i zalecaną formą wprowadzania danych jest instrukcja „INSERT”.

Zdecydowałem się na stworzenie skryptu w pythonie który na podstawie prototypowych danych generuje zapytania wykorzystujące funkcje „INSERT”. Dzięki temu wygenerowany plik pasuje do baz danych

4.3 Komentarz do procesu wprowadzania danych

1. W pierwszej kolejności zapełnione zostały tabele „filmy” oraz „sale” aby wygenerować na nich klucze główne.
2. Później dane są wprowadzane do tabeli „seanse” aby nie wystąpiła sytuacja z naruszeniem klucza obcego.

Zapytania do bazy danych

Autor: Mateusz Gałeczki

Stworzyłem funkcje w języku python obsługujące zapytania Postgres i SQLite

5.1 Lokalizacja

- **Testowany Katalog:** Plik był testowany na serwerze Jupyterhub «/home/student08» oraz Postgres «student08db1/student08@postgres»
- **Link do repozytorium:** Plik nazywa się „zapytania_kino.py” https://github.com/szefito123/Sprawozdanie-Glowne/blob/main/zapytania_kino.py

5.2 Opis funkcji

Przygotowałem po 5 zapytań dla każdej bazy danych

1. **Selekcja danych i funkcje wierszowe:** Wykorzystano funkcje UPPER() oraz SUBSTR() w SQLite do transformacji tekstu i dat oraz ``CASE WHEN`` w PostgreSQL do sprawdzania długości filmów.
2. **Funkcje agregujące:** Zastosowano operacje SUM(), COUNT(), MAX() oraz AVG() połączone GROUP BY oraz HAVING.
3. **Połączenia:** Wykorzystano (INNER JOIN) do relacji między trzema tabelami (filmy, sale, seanse) oraz złączenia lewostronne (LEFT JOIN) wraz z funkcjami obsługi wartości pustych (IFNULL() / COALESCE()) do uwzględnienia rekordów pustych.
4. **Operatory zbiorowe:** Użyłem UNION, EXCEPT oraz INTERSECT do filtrowania sal i repertuaru.

5. **Podzapytania:** Wprowadzono podzapytania skorelowane i nieskorelowane do porównywanie czasu trwania filmów ze średnią `SELECT AVG()`.

5.3 Dokumentacja kodu

Dokumentacja jest generowana automatycznie na podstawie komentarzy przez rozszerzenie Sphinx

`zapytania_kino.pg_kategoryzacja_filmow(conn_params: dict) → list`

Daje każdemu filmowi kategorie powyżej i poniżej 120 minut i pokazuje w jakiej sali był seans

`zapytania_kino.pg_najnowsze_filmy_w_malych_salach(conn_params: dict) → list`

Pokazuje najnowsze filmy które są grane w salach mających mniej niż 50 miejsc

`zapytania_kino.pg_szczegoly_najdluzszego_filmu(conn_params: dict) → list`

Pokazuje informacje o filmach których czas trwania jest większy niż globalna średnia innych filmów

`zapytania_kino.pg_wspolne_seanse_intersect(conn_params: dict) → list`

Pokazuje seanse w których film trwa ponad 150 minut oraz filmy zrobione przed 2005 rokiem

`zapytania_kino.pg_zaawansowane_zestawienie_filmow(conn_params: dict) → list`

Wyświetla tytuł, liczbę seansów, średnią pojemność sal w których był wyświetlany oraz pokazuje tylko filmy z przynajmniej jednym seansem

`zapytania_kino.sqlite_filmy_archiwalne_i_premium(db_path: str) → list`

Pokazuje liste filmów które został zrobione przed 2000 rokiem oraz były wyświetlane w sali o pojemności większej niż 100

`zapytania_kino.sqlite_pojemnosc_sal_dla_dlugich_filmow(db_path: str) → list`

Pokazuje wszystkie sale w których filmy są dłuższe niż średni czas trwania wszystkich filmów

`zapytania_kino.sqlite_puste_sale(db_path: str) → list`

Pokazuje w jakich salach nie odbył się żaden seans

`zapytania_kino.sqlite_raport_popularnosci_filmow(db_path: str) → list`

Tworzy raport zawierający tytuł, liczbę seansów i największą sale w której był pokazany

`zapytania_kino.sqlite_seanse_wielkie_litery(db_path: str) → list`

Wyświetla wszystkie seanse po 2026 roku zmieniając tytuły na wielkie litery

Indeks modułów Pythona

Z

`zapytania_kino`, [45](#)

M

module

zapytania_kino, [45](#)

P

pg_kategoryzacja_filmow() (w module
zapytania_kino), [45](#)

pg_najnowsze_filmy_w_malych_salach()
(w module zapytania_kino), [45](#)

pg_szczegoly_najdluzszego_filmu()
(w module zapytania_kino), [45](#)

pg_wspolne_seanse_intersect() (w mo-
dule zapytania_kino), [45](#)

pg_zaawansowane_zestawienie_filmow()
(w module zapytania_kino), [45](#)

S

sqlite_filmy_archiwalne_i_premium()
(w module zapytania_kino), [45](#)

sqlite_pojemnosc_sal_dla_dlugich_filmow()
(w module zapytania_kino), [45](#)

sqlite_puste_sale() (w module zapyta-
nia_kino), [45](#)

sqlite_raport_popularnosci_filmow()
(w module zapytania_kino), [45](#)

sqlite_seanse_wielkie_litery() (w
module zapytania_kino), [45](#)

Z

zapytania_kino
module, [45](#)